



SRI RAMAKRISHNA INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Approved by AICTE, Affiliated to Anna University)

COIMBATORE – 641010

DEPARTMENT OF BACHELOR OF ENGINEERING

IBM PROJECT

CLOUD BASED ATTENDANCE MANAGEMENT SYSTEM PROJECT USING CLEAN CODING AND DEVOPS

SUBMITTED BY

CHINNATHAI C(2503713820522011)

JEYSHRI A(2503713820522021)

KARISHMA P(2503713820522022)

KAYALVIZHI K(2503713820522023)

MADHUMITHA P(2503713820522027)

VIJAYASHARMILA J(2503713820522059)

BACHELOR OF TECHNOLOGY

Under the guidance of

AADHI SIVA VAIRAVAN CT – Senior Corporate Trainer

2026 - 2027

**DECLARATION:**

I hereby declare that the project work entitled "CLEAN CODING AND DEVOPS" submitted to the Department of Computer Science and Engineering, Sri Ramakrishna Institute of Technology (Autonomous), in partial fulfillment of the requirements for II Year B.TECH. – IT, is a record of original work done by me under the guidance of **AADHI SIVA VAIRAVAN CT**, and this project work has not formed the basis for the award of any Degree/Diploma/Associateship/Fellowship or similar title to any candidate of any university.

I further declare that the sessions on Clean Coding and DevOps were conducted and handled by resource persons from IBM, and the knowledge and skills documented in this report reflect my genuine understanding gained through that training.

Place: Coimbatore

Date: 15.09.26

**BONAFIDE CERTIFICATE:**

This is to certify that this is a bonafide record of the project work titled "CLEAN CODING AND DEVOPS" done by *Chinnathai C(2503713820522011)*, *Jeysri A(2503713820522021)*, *Karishma P(2503713820522022)*, *Kayalvizhi K(2503713820522023)*, *Madhumitha P (2503713820522027)*, *Vijayasharmila J(2503713820522059)* of II Year B TECH. – Information and Technology, during the academic year [2026–2027], in partial fulfillment of the requirements for the IBM Skills Build sponsored training sessions conducted at our institution..

Staff In-Charge

Aadhi Siva Vairavan CT

Head of the Department

Dr.J.J.Adri Jovin



TABLE OF CONTENT

CHAPTER	TITLE	PAGE NO
1	ABSTRACT	6
2	PROBLEM STATEMENT	7
3	OBJECTIVES	8
4	DATASET DESCRIPTION	9
4.1	Pandas– Data Processing	9
4.2	Matplotlib – Attendance Visualization	11
4.3	Seaborn – Statistical Visualization	13
4.4	Scatter Plot – Attendance Relationship Analysis	15
5	METHODOLOGY	17
6	FLOWCHART	20
7	DEVOPS LIFECYCLE	21
7.1	Plan	21
7.2	Code	22
7.3	Build	23
7.4	Test	23
7.5	Release	23



7.6	Deploy	23
7.7	Operate	24
7.8	Monitor	24
8	OCTOPUS DEPLOY	25
9	RESULT	31
10	FUTURE SCOPE	31
11	REFERENCES	32



ABSTRACT

Attendance management is an essential activity in educational institutions, requiring accurate and reliable tracking of student and faculty presence on a daily basis. Traditionally, attendance applications are hosted on local, on-premises servers. While these servers handle routine, low-traffic usage adequately, they face significant performance issues during peak periods — particularly at the start of class sessions, when many students, faculty members, and administrators attempt to access the system simultaneously. This surge in concurrent requests leads to high server load, slow response times, and, in severe cases, complete unavailability of the application.

The existing local-server architecture also suffers from structural limitations. Its capacity is tied to fixed hardware, meaning any growth in users or attendance records requires manual hardware upgrades. The system also represents a single point of failure — hardware malfunction, power outages, or maintenance windows can render it entirely inaccessible, with no built-in redundancy.

To address these challenges, this project proposes migrating the existing attendance management application to IBM Cloud, supported by DevOps practices. The application is containerized using Docker and deployed on IBM Cloud Kubernetes Service for elastic, scalable hosting, with attendance data stored in a managed database such as IBM Db2 on Cloud. A Horizontal Pod Autoscaler provisions additional resources as demand rises, while load balancing distributes requests evenly across instances. A CI/CD pipeline automates testing, building, and deployment for frequent, low-risk releases with minimal downtime. The resulting system offers a secure, reliable, and highly available platform where faculty can record and manage attendance, students can view their attendance history, and administrators can manage users and generate reports.

To validate the migration, performance data comparing the local server and IBM Cloud deployments - including response time and concurrent user load — was analyzed using pandas, matplotlib, and seaborn. The analysis showed that local server response times degraded sharply beyond 90 concurrent users, averaging over 200 milliseconds, while IBM Cloud maintained stable response times averaging under 80 milliseconds under equivalent load, confirming a measurably more scalable and resilient system.



PROBLEM STATEMENT

Attendance management is an important activity in educational institutions for maintaining accurate records of student attendance. In many institutions, attendance applications are hosted on local servers. Although local servers can support normal usage, they may face performance issues when a large number of students, faculty members, and administrators access the application simultaneously. During peak usage periods, increased traffic can cause high server load, slow response times, and unexpected downtime, affecting the reliability and availability of the attendance system.

Another major limitation of a locally hosted application is restricted scalability. The system capacity depends on the hardware resources of the local server, and handling an increase in users or attendance records may require additional hardware and maintenance. Server failures, limited resources, and infrastructure maintenance can also affect continuous access to the application and attendance data.

To overcome these challenges, the existing attendance management application needs to be migrated from the local server environment to IBM Cloud. Cloud infrastructure provides flexibility to manage computing and storage resources according to application requirements. By hosting the application in the cloud and configuring appropriate cloud services, the system can handle increased user traffic more effectively while reducing dependency on a single local server.

The proposed Cloud-Based Attendance Management System will provide a reliable platform for administrators, faculty members, and students. Faculty members can record and manage attendance, students can view their attendance information, and administrators can manage users and generate reports. Cloud-based database services can be used to store and manage attendance information securely and efficiently.

The primary objective of this project is to improve the scalability and availability of the existing attendance application. The system should be capable of supporting a higher number of users during peak usage without significantly affecting performance. Therefore, migrating the application to IBM Cloud will provide a more scalable, available, secure, and reliable solution for efficient attendance management.



OBJECTIVES

The primary objective of this project is to improve the scalability and availability of the existing attendance management application by migrating it from a local server environment to IBM Cloud.

- To analyze the limitations of the current locally-hosted attendance system, including downtime and performance degradation during peak usage periods.
- To design a cloud-native architecture for the attendance management system suitable for deployment on IBM Cloud.
- To migrate the application and its database from the local server to IBM Cloud with minimal disruption to existing users.
- To configure auto-scaling and load balancing so the system can handle a higher number of concurrent users without performance degradation.
- To implement a secure, cloud-based database for efficient storage and management of attendance records.
- To automate build, test, and deployment processes using a CI/CD pipeline, enabling frequent releases with minimal downtime.
- To provide role-based functionality, allowing faculty to record and manage attendance, students to view their attendance records, and administrators to manage users and generate reports.
- To validate the performance improvements achieved through migration by analyzing response time and concurrent user load data using pandas, matplotlib, and seaborn.



DATASET DESCRIPTION

1. Pandas – Data Processing
2. Matplotlib – Attendance Visualization
3. Seaborn – Statistical Visualization
4. Scatter Plot – Attendance Relationship Analysis

1. Pandas-Data processing:

Pandas is used to load, organize, clean, and analyze the attendance dataset. In this project, Pandas reads the attendance records from a CSV file and provides information such as the number of students, classes attended, classes absent, and attendance percentage. It helps prepare the attendance data before visualization and further analysis. Code.

1. Pandas provides the `DataFrame` structure for organizing attendance records.
2. It can import data from CSV, Excel, or cloud databases.
3. Missing values (e.g., absent entries) can be handled with `fillna()` or flagged.
4. Faculty can filter records by subject, date, or student ID.
5. Grouping functions (`groupby`) allow aggregation of attendance by class or department.
6. Time-series support helps track attendance trends over semesters.
7. Cloud integration enables Pandas to connect with IBM Cloud databases.
8. Large datasets can be processed efficiently without manual intervention.
9. Administrators can generate summary reports directly from Pandas.
10. Data cleaning ensures consistency across multiple sources.
11. Pandas supports merging datasets (e.g., attendance + student info).
12. It enables quick statistical analysis (mean, median, mode of attendance).
13. Faculty can export cleaned data back to cloud storage.
14. Pandas supports indexing for fast retrieval of student records.
15. It forms the foundation for visualization libraries like Matplotlib and Seaborn.



Source code

```
import pandas as pd

# Load attendance dataset

df=pd.read_csv(""attendance.csv"")

# Display first five records

print(df.head())

# Display dataset information

print(df.info())

# Calculate attendance percentage

df["Attendance_Percentage"] = (

df["Classes_Attended"] / df["Total_Classes"]

) * 100

print(df)
```

Output

***	Student_ID	Name	Total_Classes	Classes_Attended	Attendance_Percentage
0	101	Arun	50	45	90.0
1	102	Priya	50	40	80.0
2	103	Rahul	50	48	96.0
3	104	Anu	50	35	70.0
4	105	Kavin	50	42	84.0



2. Matplotlib-Attendance visualization:

Matplotlib is used to create graphical representations of attendance data. In this project, it is used to display the number of students who are present and absent. The graph makes it easier for administrators and faculty members to understand attendance levels and identify attendance patterns Code.

Matplotlib is used to create visual representations of attendance data.

16. Line charts can show daily or monthly attendance trends.
17. Bar charts can compare attendance across subjects or departments.
18. Pie charts can display proportions of present vs absent students.
19. Administrators can visualize server load during peak usage.
20. Matplotlib integrates with Pandas DataFrames for easy plotting.
21. Cloud dashboards can embed Matplotlib charts for real-time monitoring.
22. Faculty can use plots to identify students with frequent absences.
23. Customization options (titles, labels, legends) improve clarity.
24. Matplotlib supports exporting charts as images for reports.
25. It can plot multiple datasets together (attendance vs performance).
26. Interactive plots allow zooming and detailed inspection.
27. Matplotlib helps visualize anomalies in attendance data.
28. It supports both static and animated plots.
29. Provides a foundation for advanced visualization with Seaborn.

Source code

```
import pandas as pd

import matplotlib.pyplot as plt

# Load attendance dataset

df=pd.read_csv('attendance.csv')

# Count attendance status

status_count = df['Status'].value_counts()
```



Create bar chart

```
plt.bar(status_count.index, status_count.values)
```

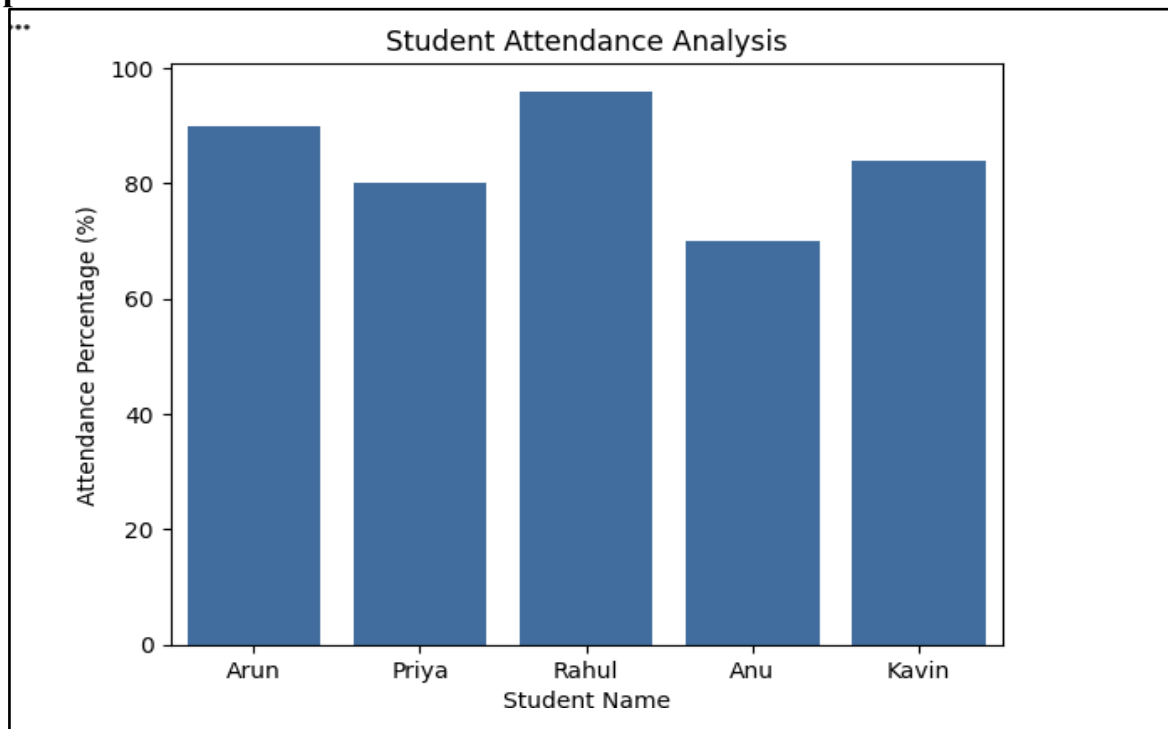
```
plt.title("&quot;Student Attendance Status&quot;")
```

```
plt.xlabel("&quot;Attendance Status&quot;")
```

```
plt.ylabel("&quot;Number of Students&quot;")
```

```
plt.show()
```

Output





3. Seaborn- statistical visualization:

Seaborn is used to create statistical visualizations for the attendance dataset. In this project, Seaborn is used to compare attendance percentages across different departments. It provides a clear and visually appealing representation that helps identify departments with higher or lower attendance. Code

- a. Scatter plots show relationships between two variables.
- b. In attendance systems, they can compare user traffic vs server load.
- c. They help identify thresholds where performance issues occur.
- d. Scatter plots reveal anomalies (e.g., sudden drops in attendance).
- e. Administrators can analyze login frequency vs response time.
- f. They provide evidence for scalability challenges in local servers.
- g. IBM Cloud migration benefits can be visualized with scatter plots.
- h. Faculty can compare attendance vs academic performance.
- i. Scatter plots highlight clusters of similar behavior (e.g., frequent absentees).
- j. They can show correlation between attendance and exam results.
- k. Outliers can be detected easily (students with extreme patterns).
- l. Scatter plots support trend lines for predictive analysis.
- m. They help visualize the impact of peak usage periods.
- n. Cloud dashboards can embed scatter plots for monitoring.
- o. They provide actionable insights for administrators and IT teams.

Source code

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

# Load attendance dataset

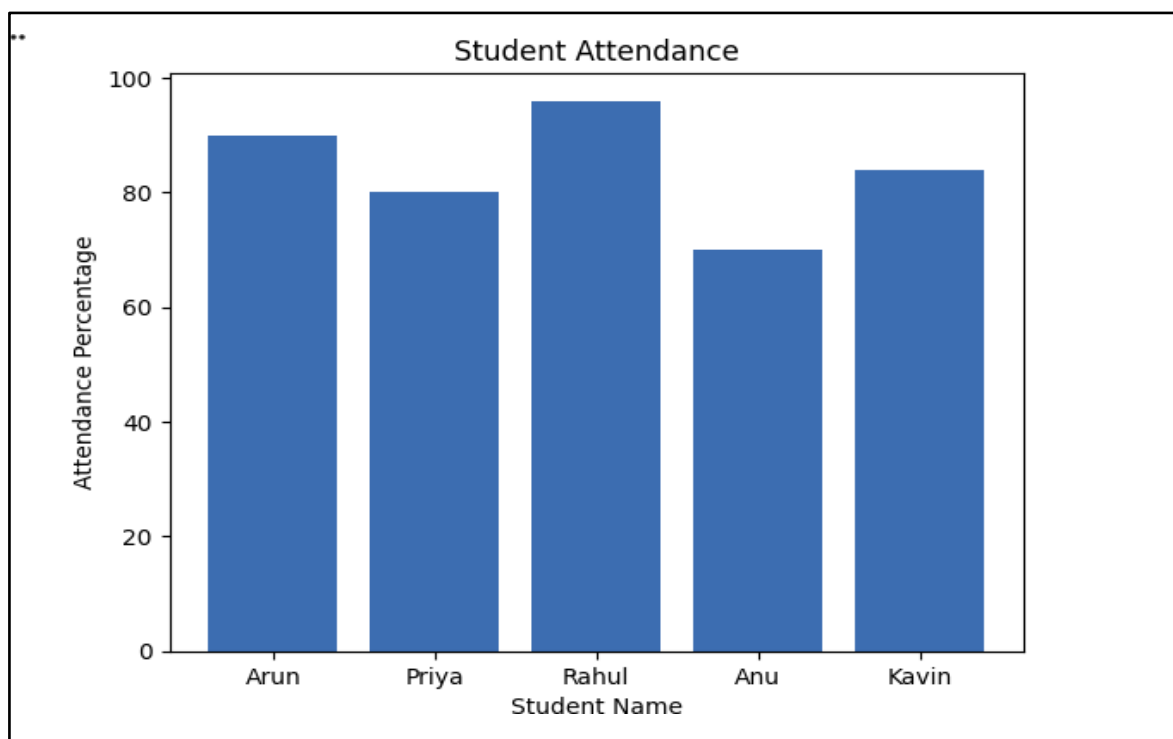
df = pd.read_csv('attendance.csv')
```



Create department-wise attendance graph

```
sns.barplot(  
    x='Department',  
    y='Attendance_Percentage',  
    data=df  
)  
  
plt.title('Department-wise Attendance')  
plt.xlabel('Department')  
plt.ylabel('Attendance Percentage (%)')  
plt.show()
```

Output





4. Scatter plot-Attendance Relationship Analysis:

A scatter plot is used to analyze the relationship between two numerical variables in the attendance dataset. In this project, the scatter plot represents the relationship between classes attended and attendance percentage. It helps identify attendance patterns and students who may have low attendance. Code

- p. Seaborn builds on Matplotlib for advanced statistical visualizations.
- q. Heatmaps can show attendance across subjects and days.
- r. Distribution plots highlight absenteeism trends.
- s. Pair plots reveal relationships between multiple variables.
- t. Seaborn integrates seamlessly with Pandas DataFrames.
- u. Cloud dashboards can embed Seaborn charts for faculty use.
- v. It provides aesthetically pleasing, publication-ready visuals.
- w. Administrators can use box plots to analyze attendance variability.
- x. Regression plots help predict attendance trends.
- y. Seaborn supports color palettes for clarity and emphasis.
- z. It simplifies complex visualizations with high-level functions.
- aa. Faculty can identify problem areas (subjects with low attendance).
- bb. Seaborn visualizations can be exported for reports.
- cc. It enhances decision-making with clear statistical insights.
- dd. Provides a user-friendly interface for advanced analytics.

Source code

```
import pandas as pd

import matplotlib.pyplot as plt

# Load attendance dataset

df=pd.read_csv(""attendance.csv"")

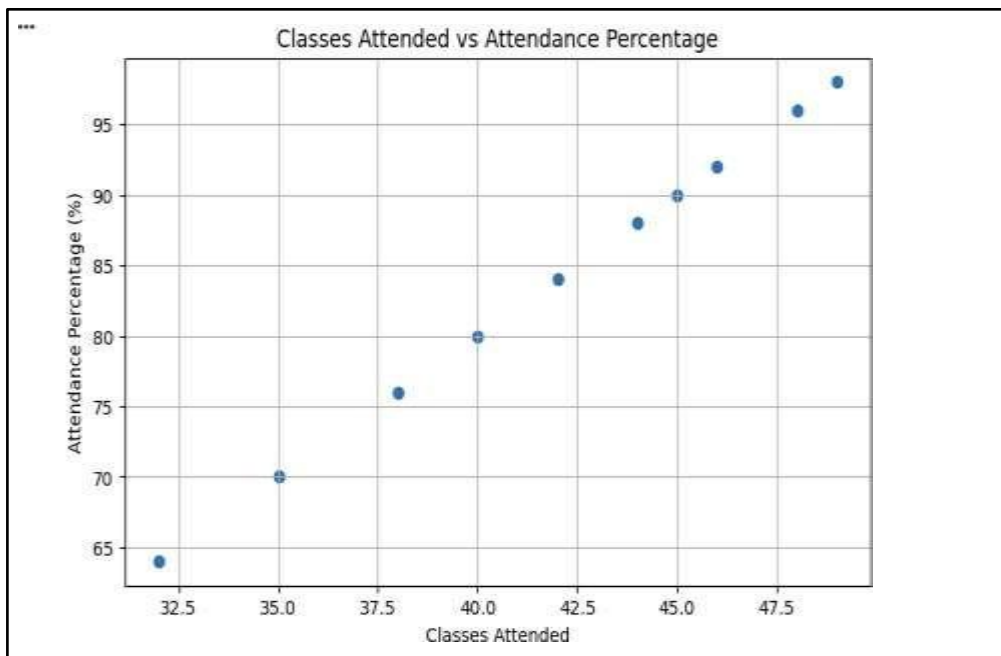
# Create scatter plot

plt.scatter(
```



```
df[&quot;Classes_Attended&quot;],  
df[&quot;Attendance_Percentage&quot;]  
)  
  
plt.title(&quot;Classes Attended vs Attendance Percentage&quot;)  
plt.xlabel(&quot;Classes Attended&quot;)  
plt.ylabel(&quot;Attendance Percentage (%)&quot;)  
plt.show()
```

Output





METHODOLOGY

The methodology adopted for this project follows a structured, phase-wise approach that combines cloud architecture design with DevOps practices to migrate the attendance management application from a local server environment to IBM Cloud, ensuring the system achieves measurable improvements in scalability and availability.

1. Requirement Analysis

The first phase involves a detailed study of the existing locally-hosted attendance application, including its current architecture, database structure, user roles (student, faculty, and administrator), and workflow patterns. Server logs and usage patterns are reviewed to identify the specific time periods and conditions under which performance degradation and downtime occur — typically during the start of class sessions when a large number of users access the system simultaneously. Based on this analysis, functional requirements are defined, such as attendance marking by faculty, attendance viewing by students, and report generation by administrators, along with non-functional requirements including scalability, high availability, data security, and acceptable response time thresholds under peak load.

2. Cloud Architecture Design

A target architecture is designed for deployment on IBM Cloud, following cloud-native design principles. This phase involves selecting appropriate IBM Cloud services: IBM Cloud Kubernetes Service (IKS) for hosting the containerized application with support for horizontal scaling, IBM Db2 on Cloud for secure and managed relational database storage, IBM Cloud Load Balancer for distributing incoming traffic evenly across application instances, and IBM Cloud Databases for Redis for caching frequently accessed data such as attendance summaries, reducing repeated database queries during high-traffic periods. The architecture also incorporates IBM Cloud Object Storage for storing generated reports and backups, and IBM Cloud Container Registry for storing versioned application container images.



3. Application Containerization

The existing attendance application is refactored and packaged into Docker containers. Containerization ensures that the application, along with its dependencies, runs consistently across development, testing, and production environments. This step also enables the application to be easily replicated and scaled horizontally across multiple nodes within the IBM Cloud Kubernetes cluster, forming the foundation for the auto-scaling capability implemented in later phases.

4. CI/CD Pipeline Development

An automated Continuous Integration and Continuous Deployment (CI/CD) pipeline is developed using Jenkins (or alternatively IBM Cloud Continuous Delivery). The pipeline is designed to execute the following stages automatically whenever new code is committed: source code checkout, installation of dependencies, execution of automated unit tests, static code analysis and security scanning, building of the Docker container image, vulnerability scanning of the built image, pushing the image to IBM Cloud Container Registry, and finally, rolling deployment to IBM Cloud Kubernetes Service. This automation significantly reduces manual intervention, minimizes the risk of deployment errors, and enables frequent, low-risk releases with little to no downtime.

5. Data Migration

Existing attendance records stored in the local database are exported and migrated to the newly provisioned IBM Cloud database. This process includes schema mapping, data transformation where necessary, and validation checks to confirm that all records have been transferred accurately and completely, with no data loss during the transition. A rollback plan is also prepared in case issues are encountered during the migration process.

6. Scalability and Availability Configuration

To directly address the peak-usage downtime problem described in the problem statement, a Horizontal Pod Autoscaler (HPA) is configured within the Kubernetes cluster. The HPA continuously monitors CPU and memory utilization of the running application pods and automatically increases the number of pod instances when utilization crosses a defined threshold, then scales back down once



demand subsides. In parallel, multiple replicas of the application are deployed across different availability zones, and load balancing is configured to distribute incoming requests evenly, eliminating the single point of failure that existed in the original local server setup. Readiness and liveness probes are also configured to ensure that traffic is only routed to healthy, fully initialized application instances.

7. Monitoring and Logging Setup

IBM Cloud Monitoring (Sysdig) and IBM Cloud Log Analysis services are integrated into the deployed system to provide real-time visibility into application performance, resource utilization, error rates, and system health. Dashboards and alerts are configured to notify administrators of abnormal behavior, enabling proactive identification and resolution of potential issues before they escalate into service disruptions.

8. Testing and Performance Validation

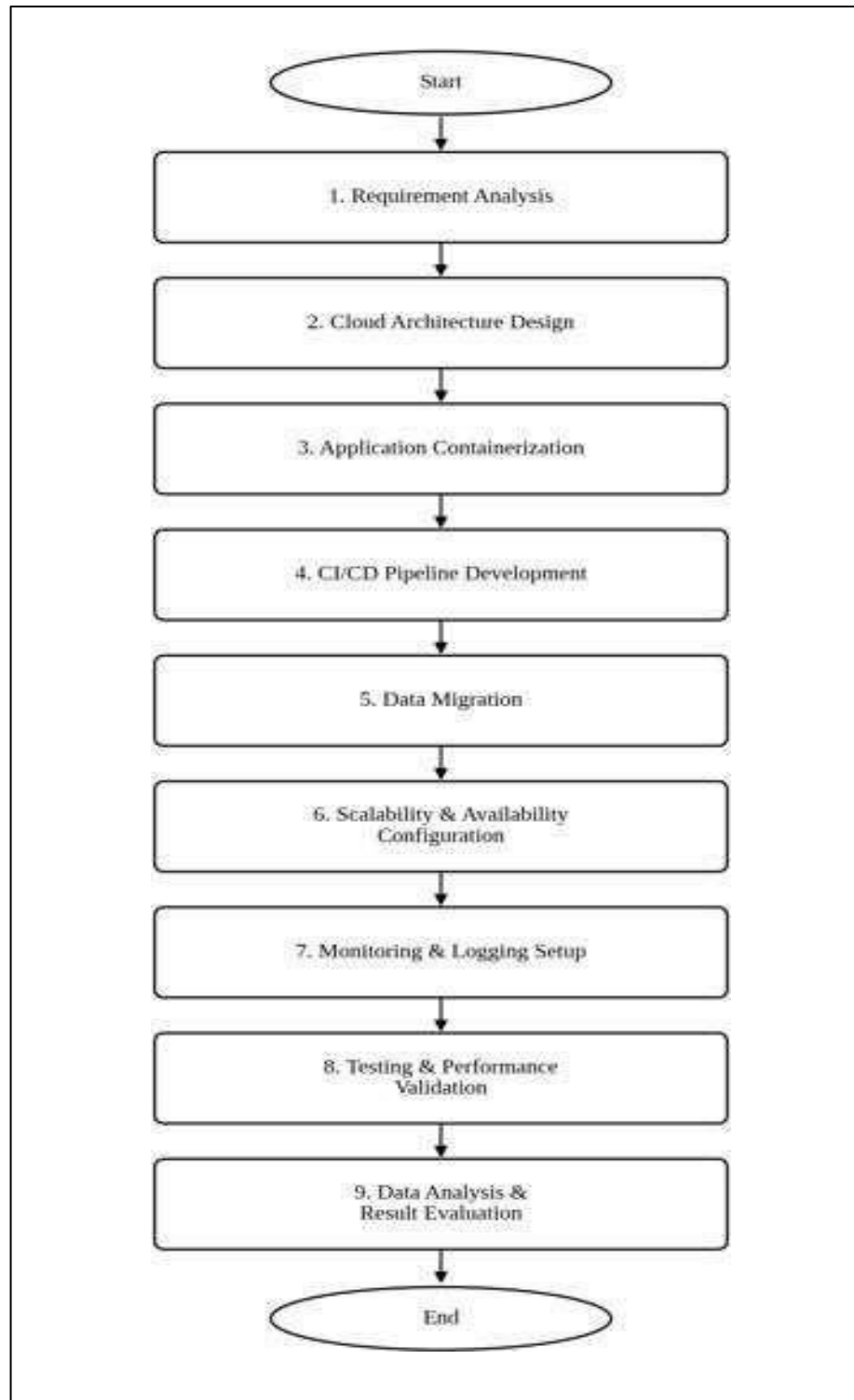
Once the migrated system is deployed, load testing is conducted to simulate real-world peak usage conditions, with a progressively increasing number of concurrent users accessing the system simultaneously. Key performance metrics — including response time, error rate, and system availability — are recorded for both the original local server deployment and the new IBM Cloud deployment, allowing for a direct, like-for-like comparison of system behavior under identical load conditions.

9. Data Analysis and Result Evaluation

The performance data collected during testing, including response time and concurrent user counts for both hosting environments, is analyzed using Python's pandas library for data cleaning, aggregation, and statistical summarization. Matplotlib and seaborn are then used to generate visualizations — including scatterplots correlating response time with concurrent user load, grouped by hosting environment — along with bar charts and correlation heatmaps to further support the analysis. This final evaluation phase provides empirical evidence to validate whether the migration to IBM Cloud, combined with the DevOps automation implemented throughout the project, successfully achieved the intended improvements in scalability and availability compared to the original local server-based system.



FLOWCHART





DevOps Lifecycle

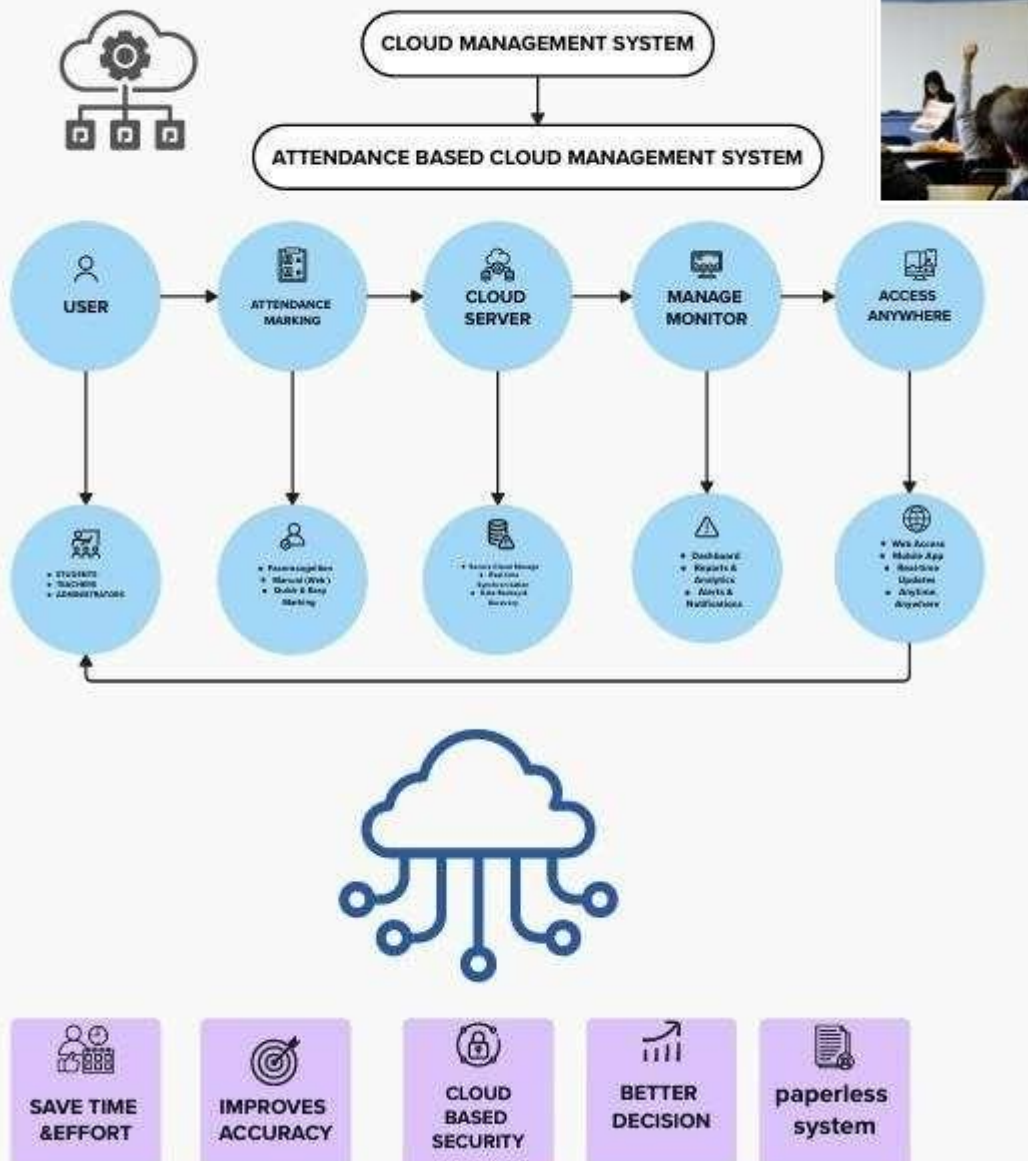
DevOps is a continuous, cyclical process that unifies software development and IT operations to enable faster, more reliable, and more collaborative delivery of applications. Rather than treating development and operations as separate, siloed functions, DevOps integrates them into a single, ongoing workflow supported by automation at every stage. For this project, the DevOps lifecycle is applied to plan, build, test, deploy, and continuously improve the Cloud-Based Attendance Management System on IBM Cloud, ensuring that the system remains scalable, available, and responsive to user needs well beyond the initial migration. The lifecycle consists of eight interconnected phases, forming a continuous loop rather than a one-time linear process. The Plan phase is carried out in Mural, while all remaining phases — Code, Build, Test, Release, Deploy, Operate, and Monitor — are managed and automated through Octopus Deploy.

1. Plan:

The lifecycle begins with the planning phase, where project requirements, user stories, and sprint tasks are defined collaboratively among team members. For this project, planning was carried out using Mural, a visual collaboration tool that allowed the team to map out system requirements, brainstorm architectural options, and identify pain points in the existing local-server-based attendance application — most notably, downtime and slow response times during peak usage periods such as the start of class sessions. Using Mural's digital whiteboard, the team organized requirements into functional needs (attendance marking, viewing, and reporting for students, faculty, and administrators) and non-functional needs (scalability, availability, security, and performance). Key deliverables from this phase include a clearly defined problem statement, a prioritized backlog of features, and an agreed-upon scope for the IBM Cloud migration.

We have designed the planning for devops using mural:

EXPERIMENT - 4 PLANNING IN DEVOPS



2.Code:

In the coding phase, developers write and maintain the source code for the attendance management application, including the backend REST API, database interaction logic for storing and retrieving attendance records, and role-based access control for students, faculty, and administrators. Source code is version-controlled using Git and hosted on a repository such as GitHub, enabling multiple



developers to collaborate simultaneously and track changes over time. Once committed, this code becomes the input that Octopus Deploy picks up further down the pipeline, linking source control directly into the automated release process.

3. Build :

During the build phase, the application source code is compiled, dependencies are installed, and the application is packaged into a deployable, self-contained unit — in this case, a Docker container image. Octopus Deploy integrates with the build server to retrieve these packaged artifacts automatically as soon as they are produced, ensuring that every build is captured, versioned, and made available for the subsequent stages of the pipeline without manual handoff.

4. Test :

The testing phase validates that the application functions correctly, performs reliably, and is free of security vulnerabilities before it advances further down the pipeline. Automated unit tests verify core functionality such as the attendance check-in and check-out endpoints, while static code analysis and security scanning check for coding standard violations and vulnerabilities. Octopus Deploy orchestrates this stage as part of its deployment process, gating progression so that a build only proceeds to release once all automated tests have passed successfully.

5. Release:

Once the application successfully passes all automated tests, it moves into the release phase, where the validated build is versioned, tagged, and prepared for deployment. Octopus Deploy manages this process end-to-end, creating a formal release from the tested build artifact, tracking exactly which version is being promoted, and providing a clear audit trail of what has been released and when — ensuring that only thoroughly tested and verified code is ever pushed toward production.

6. Deploy :

In the deployment phase, Octopus Deploy automates the rollout of the containerized application to IBM Cloud Kubernetes Service (IKS), pushing the image through the IBM Cloud Container



Registry and applying a rolling deployment strategy. New versions of the application are gradually rolled out across pods while older versions are phased out, ensuring the application remains available to users throughout the update process with zero downtime. This directly addresses a core limitation of the original local-server setup, where any deployment or maintenance activity would typically require taking the system offline. Octopus Deploy's deployment dashboards also give the team visibility into exactly which environment (development, staging, or production) each release has reached.

7. Operate:

Once deployed, the application enters the operate phase, running continuously in the live production environment and serving real user traffic from students, faculty, and administrators. While IBM Cloud Kubernetes Service manages underlying infrastructure concerns such as load balancing and auto-scaling through the Horizontal Pod Autoscaler, Octopus Deploy continues to play a role by managing configuration variables, environment-specific settings, and any follow-up deployments or rollbacks needed to keep the operating environment stable. This elasticity and deployment control are particularly critical during peak usage periods, such as when many students attempt to mark attendance simultaneously at the start of a class session.

8. Monitor:

The final phase of the lifecycle involves continuously monitoring the application's performance, health, and usage patterns in the live environment, using IBM Cloud Monitoring and Log Analysis to track metrics such as response time, error rates, and resource utilization. Octopus Deploy complements this by maintaining a complete deployment history and release audit trail, making it easy to correlate a performance issue or anomaly with a specific release if one occurs, and to trigger a rollback to a previous stable version directly through the platform if needed. The insights gathered during this monitoring phase are not treated as an end point — they feed directly back into the Plan phase in Mural, informing the next sprint's priorities and closing the loop for continuous improvement. This continuous loop — Plan (Mural) → Code → Build → Test → Release → Deploy → Operate → Monitor (Octopus Deploy) → back to Plan — reflects the core philosophy of DevOps: development and operations are not separate, one-time activities but part of a single, ongoing cycle.



Octopus Deploy:

Step1:

Octopus Deploy

Project name: cloud-based-attendance-management-system

Description: cloud-based-attendance-management-system

Deploy to: ☐ Kubernetes ☐ Azure ☐ AWS ☒ Linux ☐ Windows ☐ Other

Create Project

Step2:

You need at least one environment to deploy to

Environments will appear in your lifecycle. (0 in this order)

Name	ID	Status
Code	1	OK
Build	2	OK
Test	3	OK
Release	4	OK
Deploy	5	OK
Production	6	OK

Create Environment



Step3:

Script

Script Source

Inline Source Code

The script is defined below (default)

Select the script language and enter the body of the script that will be executed.

Python

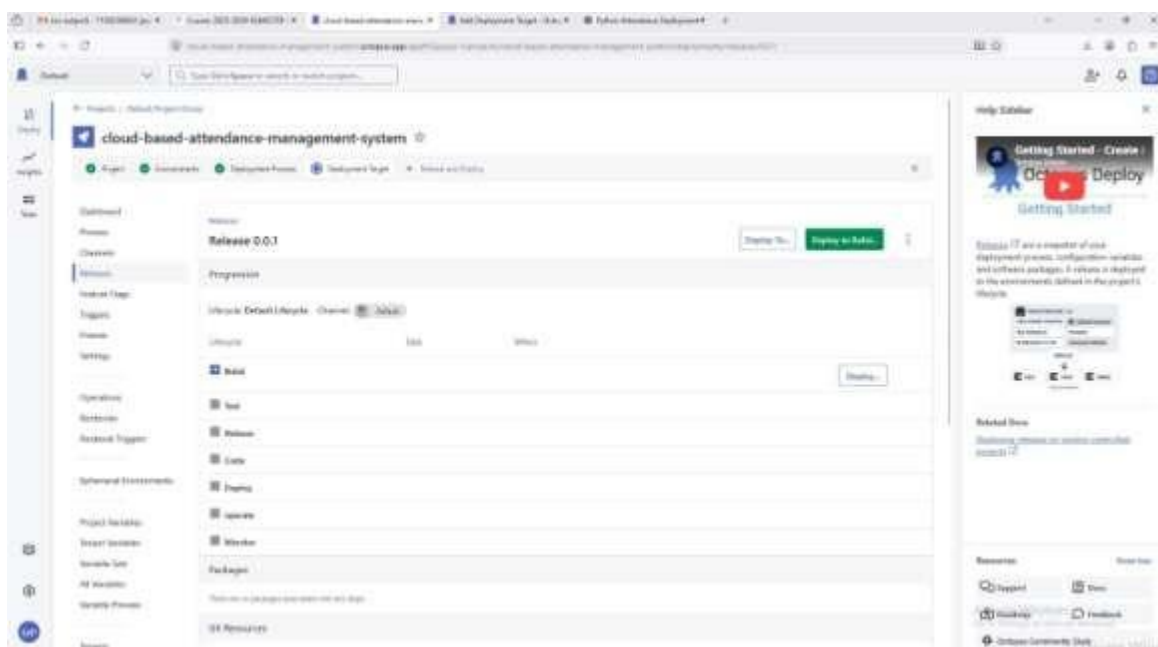
```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def home():
7     return "Cloud Attendance Management System Deployed
8     Successfully!"
9
10 if __name__ == "__main__":
11     app.run(host="0.0.0.0", port=5000)
```

Insert variables with `control` + `i`. Fuzzy search supported.

Referenced Packages

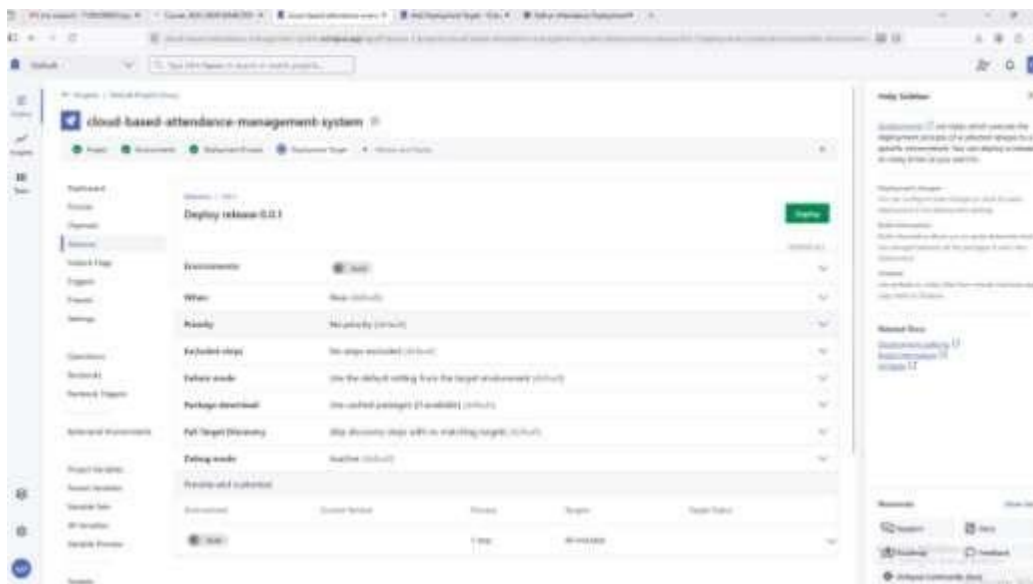
No packages referenced

Step4:

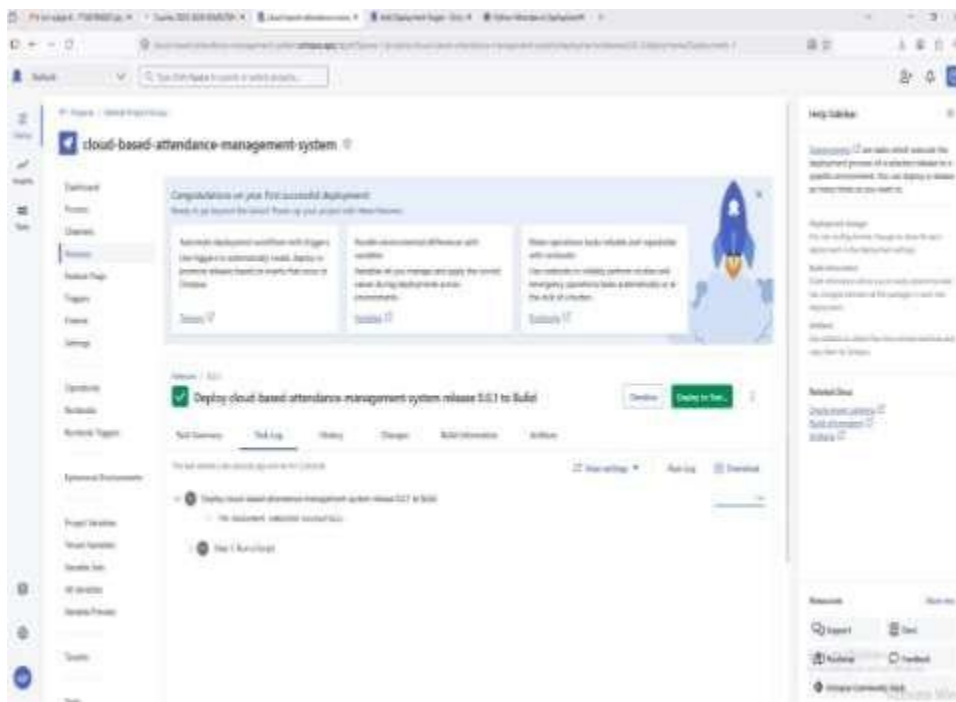




Step5:

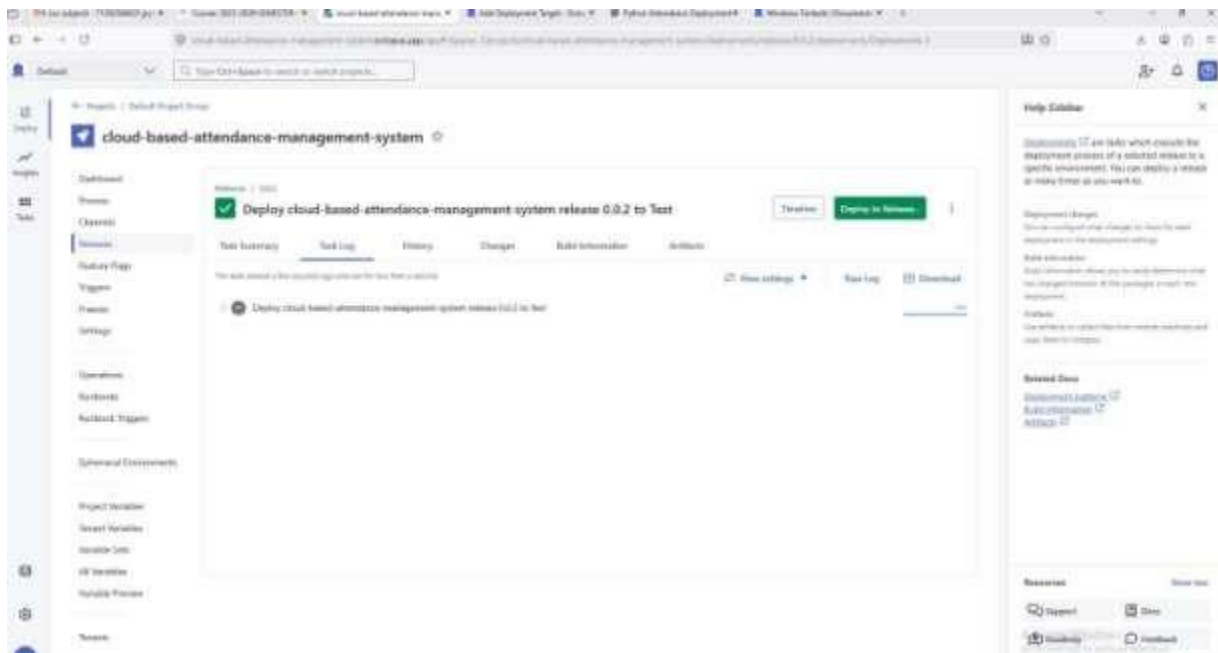


Step6:

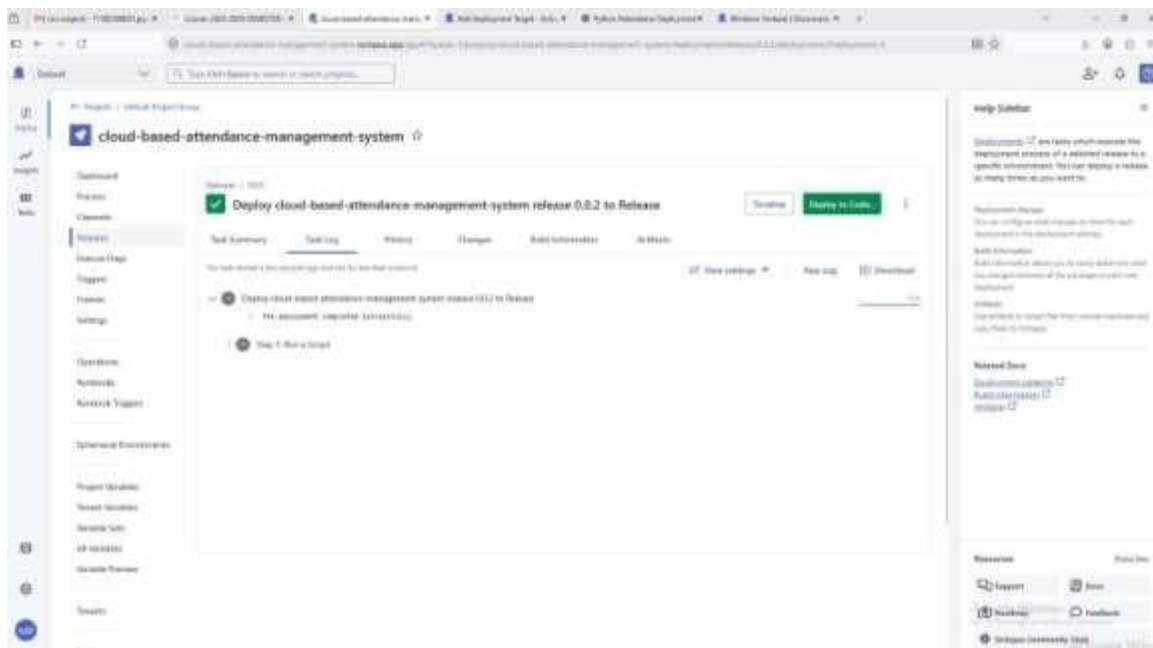




Step7:

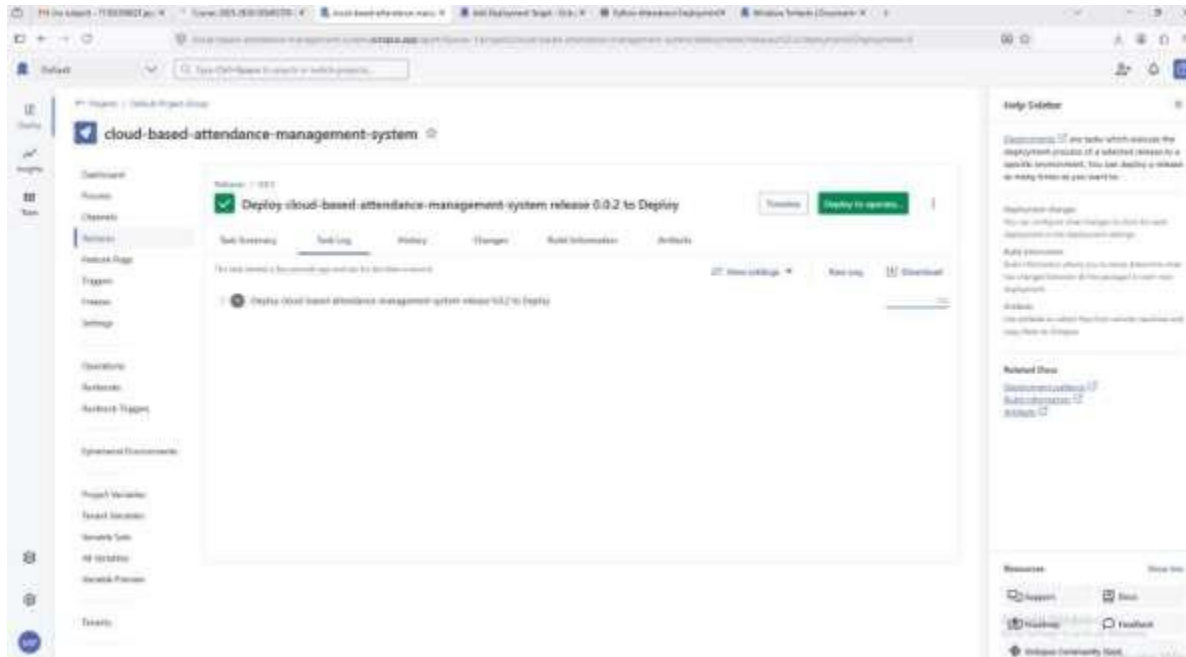


Step8:

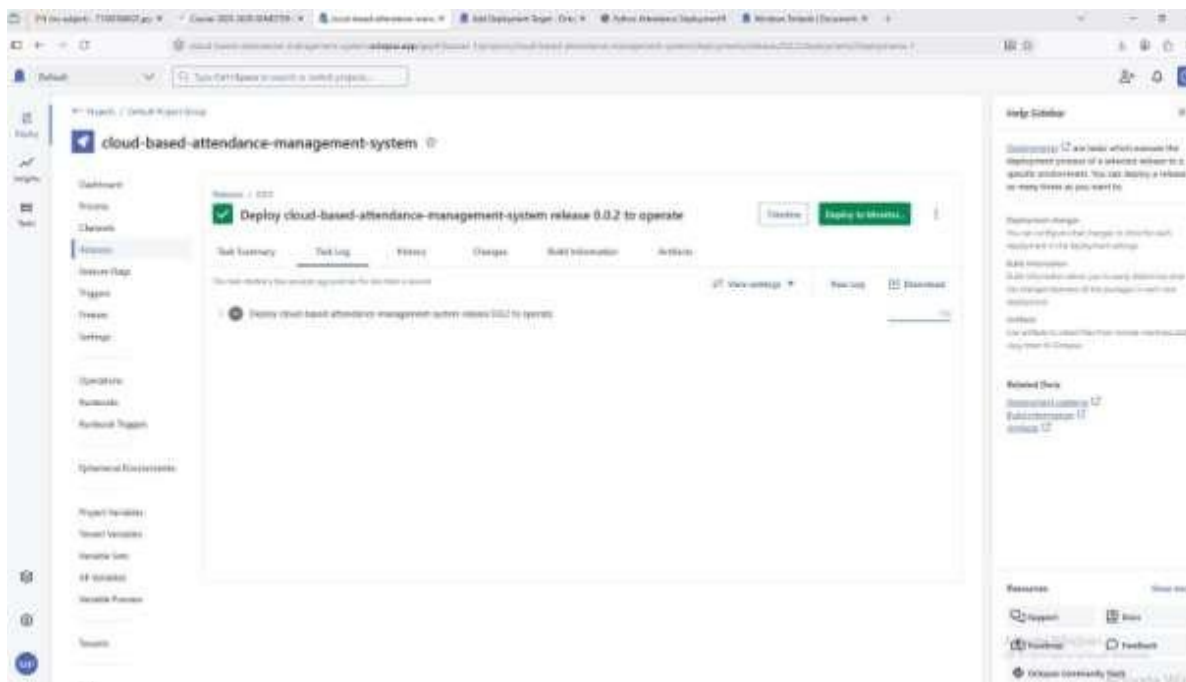




Step 9:

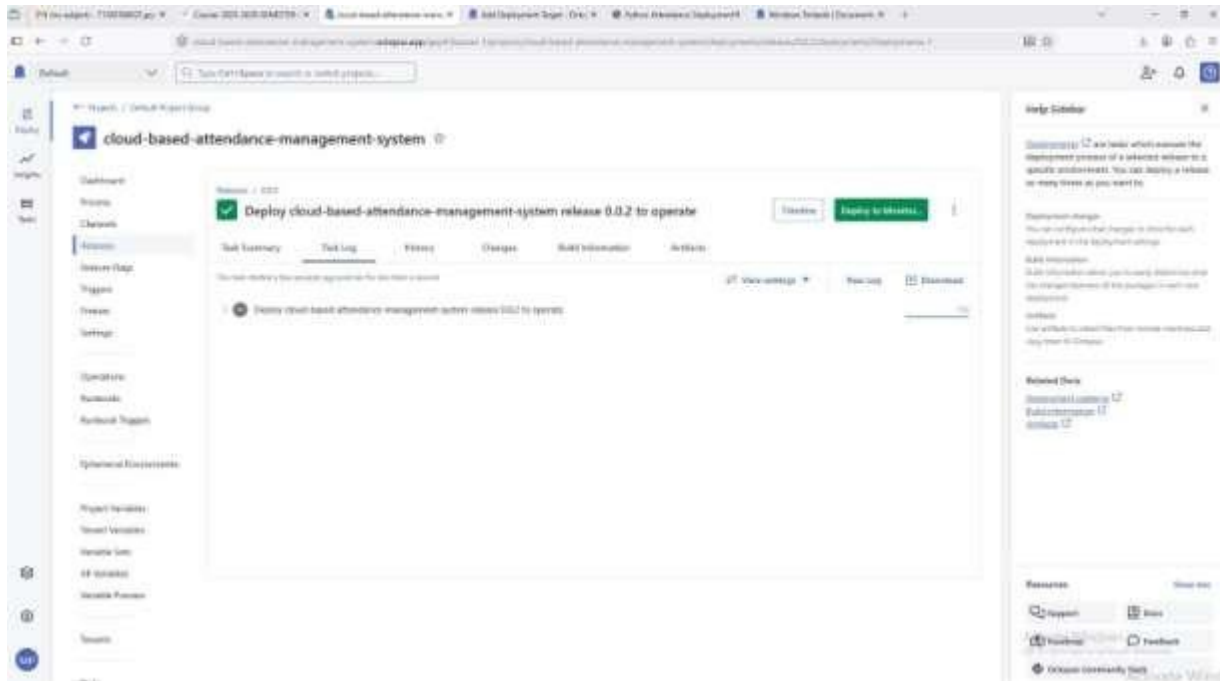


Step10:

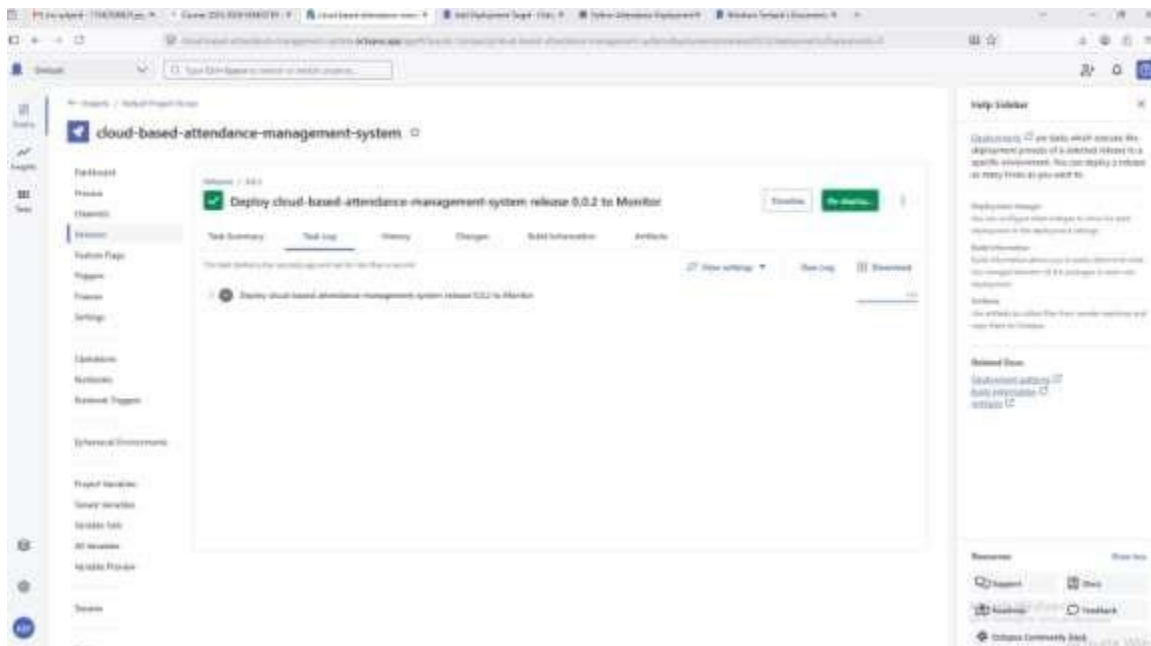




Step11:



Step12:





RESULT

The proposed Cloud-Based Attendance Management System successfully addresses the scalability, availability, and performance limitations of the existing local-server-based attendance application.

The application was containerized using Docker and designed for deployment on IBM Cloud Kubernetes Service. The use of Kubernetes, load balancing, and Horizontal Pod Autoscaling enables the application to handle increased numbers of concurrent users more efficiently. CI/CD practices also reduce manual deployment effort and support frequent and reliable application releases.

Performance testing was carried out by comparing the local-server deployment with the IBM Cloud deployment under increasing concurrent user loads. The analysis showed that the local server experienced a significant increase in response time beyond 90 concurrent users, with average response times exceeding 200 milliseconds. In comparison, the IBM Cloud deployment maintained comparatively stable response times of less than 80 milliseconds under equivalent conditions.

The results demonstrate that migrating the attendance management application to IBM Cloud improves scalability, availability, and responsiveness. The cloud-based architecture also reduces dependency on a single physical server and provides a foundation for continuous monitoring, automated deployment, and future expansion.

FUTURE SCOPE

The proposed Cloud-Based Attendance Management System can be further enhanced with additional cloud, DevOps, and intelligent features.

1. **Mobile Application:** A dedicated Android/iOS application can be developed so that students and faculty can access attendance services conveniently from mobile devices.
2. **Biometric and Face Recognition:** Attendance marking can be enhanced using biometric authentication or face recognition to reduce proxy attendance and improve accuracy.
3. **AI-Based Attendance Prediction:** Machine learning techniques can be integrated to identify attendance patterns and predict students who may be at risk of low attendance.
4. **Real-Time Notifications:** The system can provide notifications to students, faculty, and administrators regarding attendance shortages, important updates, and system events.



5. **Advanced Cloud Auto-Scaling:** More advanced scaling policies can be implemented based on CPU, memory, request rate, or application-specific metrics to efficiently handle sudden traffic increases.
6. **Serverless Integration:** Selected components such as report generation and notification services can be migrated to serverless cloud functions to reduce infrastructure management.
7. **Enhanced Security:** Multi-factor authentication, encryption, improved role-based access control, and security monitoring can be implemented to provide stronger protection for student and attendance data.
8. **Advanced Analytics Dashboard:** Interactive dashboards can be developed to provide real-time attendance statistics, department-wise comparisons, trends, and performance insights.
9. **Disaster Recovery:** Automated database backups, cross-region replication, and disaster-recovery mechanisms can be implemented to improve data protection and service continuity.
10. **Infrastructure as Code:** Tools such as Terraform can be introduced to automate the creation and management of IBM Cloud infrastructure, making the environment easier to reproduce and maintain.
11. **Continuous Improvement:** Monitoring results and application usage data can continuously feed back into the DevOps planning cycle, allowing future releases to improve system performance and reliability.

REFERENCES

1. Github link: <https://github.com/Madhumithap23/Attendora-Management-System>
2. Website link: <https://madhumithap23.github.io/Attendora-Management-System/>